

Project Retrospective: The Evolution of an Intelligent Railway Enquiry System

Introduction: The Initial Mandate

This project began with a deceptively simple goal: to build a Retrieval-Augmented Generation (RAG) system capable of answering questions from a single text file containing Indian Railways schedules. The initial mandate was a classic text-retrieval task, envisioned as a straightforward application of modern AI frameworks.

However, this project quickly evolved into a comprehensive journey through the practical complexities of modern RAG, forcing us to confront the fundamental conflict between the semantic fuzziness of AI and the deterministic precision required by real-world data. The initial path, paved with high-level libraries, led to a series of critical failures that forced a fundamental rethinking of our strategy. This document chronicles that evolution—from a naive vector-search model to a sophisticated, multi-tool native agent—highlighting the key challenges, architectural pivots, and hard-won lessons that defined its development.

1. The Naive Approach: Standard Vector RAG

Our initial strategy was conventional, relying on a standard RAG pipeline. Using established libraries like LangChain and the ChromaDB vector store, the plan was to split the source text file into manageable chunks, create numerical embeddings for each, and use vector similarity search to find the most relevant context for a user's query.

The first user-driven refinement was a move away from arbitrary character-based chunking. To maintain data integrity, we implemented **semantic chunking**, mandating that each chunk represent one complete train schedule. This was achieved by using the source file's delimiter via `text.split('')`, ensuring that the context provided to the language model was always complete.

The "Exact Number Problem"

Despite this refinement, the model suffered from a critical and immediate failure: the **"Exact Number Problem."** While vector search was effective for conceptual queries (e.g., "trains from Mumbai to Delhi"), it consistently failed to retrieve specific train schedules when queried by an exact number. A search for train `57254` would often return schedules for `57255` or `47254`. The embedding model, designed for conceptual understanding, treated these distinct numbers as semantically identical "five-digit train numbers," rendering it useless for precise lookups.

This failure exposed a fundamental truth: vector search is a tool for semantic similarity, not identity. It cannot be trusted as a substitute for a primary key lookup in a database. This limitation made it clear that a purely semantic approach was unworkable and necessitated our first major architectural redesign.

2. The First Pivot: A "Vanilla" Hybrid Search Engine

To solve the "Exact Number Problem," we made the strategic decision to abandon high-level frameworks and gain precise control over the retrieval logic. We transitioned to a "vanilla" Python implementation, leveraging PyTorch and local models, which allowed us to build a **Hybrid Search** engine that combined deterministic and semantic retrieval—a strategy unavailable in the standard LangChain RAG chain.

The new architecture was built on two complementary strategies:

Strategy 1: Exact Match	Strategy 2: Vector Search
Implementation: A simple Python dictionary (<code>train_index</code>) was created to map every unique train number directly to its corresponding text chunk.	Implementation: The existing semantic search capability was retained as a fallback mechanism.
Purpose: To provide instant, 100% accurate retrieval for any query containing a specific train number. This completely solved the "Exact Number Problem" with $O(1)$ efficiency.	Purpose: To handle vague, descriptive, or conceptual queries where an exact match would be impossible (e.g., "morning trains to Delhi").

This hybrid approach, however, introduced a new set of technical challenges related to scaling and reliability:

- **API Rate Limits:** While attempting to generate embeddings for over 13,000 train schedule chunks using the Gemini API, we were immediately throttled by `HTTP 429` (Too Many Requests) errors, halting the ingestion process.
- **Solution - Local Embeddings:** We made the architectural decision to decouple the embedding process from the query pipeline entirely. By integrating the `sentence-transformers` library, we brought the embedding process offline, using a local model to generate all embeddings on our own hardware. This eliminated API errors and costs and significantly accelerated data processing.

While this hybrid engine successfully resolved the issues of accuracy and reliability, it exposed a more subtle flaw in the semantic search component, pushing us toward further refinement.

3. Refining the Engine: Tackling the "Keyword Gap"

With the hybrid system in place, our architecture was tested again, revealing a weakness we termed the **"Keyword Gap."** This exposed a critical flaw in our semantic retrieval assumption: the embedding model's vocabulary was misaligned with our domain's entity importance. When users queried by train name (e.g., "Vijayawada Bhadrachalam Passenger"), the search returned incorrect results like "Indore Mhow Passenger," giving more semantic weight to the generic word "Passenger" than to the critical proper nouns "Vijayawada" and "Bhadrachalam."

To solve this, we implemented two key upgrades to the vector search component:

1. **Enriched Chunking:** We created a "search version" of each chunk, distinct from the raw text used for display. Before embedding, we prepended a descriptive header to each schedule (e.g., *"Train 57254 named Vijayawada Bhadrachalam Passenger runs from Vijayawada to Bhadrachalam..."*). This forced the embedding model to focus on the most important entities by placing them at the forefront of the text.
2. **Upgraded Embedding Model:** We replaced the initial `all-MiniLM-L6-v2` model with `BAAI/bge-small-en-v1.5`. The older MiniLM model is a generalist trained for broad sentence similarity. In contrast, the BGE model is a modern specialist, specifically trained for dense retrieval tasks like RAG. It naturally assigns a higher "semantic weight" to unique entities, making it far more accurate for searching our dataset by name.

With a truly robust and accurate search system now operational, the project's focus shifted from simple retrieval to building a more advanced, intelligent agent on top of this foundation.

Briefing: Development of a Hybrid Agentic RAG System for Railway Information

Executive Summary

This document outlines the iterative development and architectural evolution of an AI-powered railway enquiry system, transitioning from a basic Retrieval-Augmented Generation (RAG) model to a sophisticated, real-time, database-backed agent. The initial approach, relying solely on vector search over a raw text file of train schedules, proved inadequate for handling the structured and numerical nature of the data. Key challenges, such as the failure to retrieve exact train numbers and the misinterpretation of user queries, led to a strategic pivot.

The core innovation was the development of a **Hybrid Search Router**, a multi-layered system that prioritizes precision over probabilistic search. This architecture combines exact ID matching, keyword filtering, and semantic vector search as a fallback, ensuring accuracy for specific queries. To overcome technical hurdles like API rate limits and slow processing, the system was optimized by shifting to local, GPU-accelerated embeddings and implementing a save/load mechanism for the vector index.

Ultimately, the limitations of a static file-based system prompted a final evolution towards an **Agentic Architecture**. This model equips a Large Language Model (Gemini 1.5 Flash) with a "toolbox" of specialized functions. An "Analyst" tool uses a structured Pandas DataFrame for high-precision database queries (e.g., listing all trains between two cities), while a "Librarian" tool uses vector search for descriptive queries. This "Database-Backed Agentic AI" represents the final, production-grade solution, intelligently routing user intent to the correct tool and data source—a local database for static information and real-time APIs for live data like delays and PNR status.

1. Initial RAG Implementation and Core Challenges

The project began with a standard RAG workflow designed to answer queries based on a single text file (`All_Trains_Cleaned.txt`). The process involved:

1. **Loading:** Opening and reading the document.
2. **Chunking:** Splitting the document's text into smaller, manageable pieces.
3. **Embedding:** Converting text chunks into numerical vectors using models from OpenAI or Google (Gemini).
4. **Storing:** Saving these vectors in a database (ChromaDB).
5. **Retrieving:** Using vector similarity (cosine similarity) to find the most relevant chunks based on a user's query.
6. **Generating:** Passing the retrieved chunks and the query to an LLM to generate a human-readable answer.

This initial implementation quickly exposed fundamental limitations when applied to the highly structured nature of train schedules.

The "Exact Number" Problem

Vector search models excel at understanding semantic concepts but struggle with precise, distinct numerical identifiers.

- **Issue:** When querying for a specific train number like "57254", the embedding model perceived it as conceptually similar to "57255" or "47254".
- **Result:** The system would often fail to retrieve the correct schedule, especially when only the top 3 results were requested from a dataset of over 13,000 similar-looking

entries. The correct train might be ranked fourth or fifth in similarity and would never be seen by the LLM.

The "Keyword Gap"

The embedding models gave insufficient weight to critical keywords like city names, prioritizing common structural words instead.

- **Issue:** For a query like "Vijayawada Bhadrachalam Passenger train no?", the `all-MiniLM-L6-v2` model focused more on the word "Passenger" and the general table structure.
- **Result:** The system incorrectly retrieved the "Indore Mhow Passenger" because its vector was mathematically closer, demonstrating a failure to prioritize unique, defining entities within the text.

2. Evolution to a Hybrid Search Strategy

To overcome the flaws of pure vector search, a multi-layered, deterministic search strategy was developed. This "Hybrid Router" prioritizes accuracy by selecting the best tool for the query before falling back on semantic search.

The Three-Layer Router Architecture

The system processes queries in a specific order of priority:

1. **Layer 1: Exact ID Match (The "Sniper"):** Uses regular expressions (`re.search(r"\b(\d{5})\b", query)`) to detect if the query contains a 5-digit train number. If a match is found, it performs a direct lookup in a pre-built dictionary (`id_index`), guaranteeing 100% accuracy and bypassing vector search entirely.
2. **Layer 2: Keyword Match (The "Filter"):** If no ID is found, the system searches for keyword matches within train names. This solves queries where the user provides a name (e.g., "Vijayawada Bhadrachalam") but not the number. This layer also incorporates fuzzy matching using Python's `difflib` to handle typos (e.g., "Passenge" correctly finds "Passenger").
3. **Layer 3: Semantic Vector Search (The "AI Fallback"):** This layer is only used if the first two layers fail to find a match. It is reserved for vague, conceptual queries like "Morning trains to Delhi."

Data Preparation for Improved Retrieval

To enhance the accuracy of the vector search layer, two key data preparation techniques were implemented:

- **Enriched Chunking:** A descriptive header (e.g., "Train 57254 named Vijayawada Bhadrachalam Passenger...") was prepended to each chunk before embedding. This forced the embedding model to focus on the most important entities (number and name) rather than the table structure.
- **Flattening for Search:** To allow searching by intermediate stations (e.g., "Kamakhya"), all station names were extracted from each schedule table and "flattened" into the searchable text. This created two versions of each chunk: a "Search Version" with all keywords for the vector model and a "Display Version" with the clean, original table for the LLM to read.

3. Technical Hurdles and Optimization

The development process encountered several technical challenges related to API limitations, performance, and software dependencies.

API Rate Limits and the Shift to Local Embeddings

Initial attempts to embed over 13,000 train schedules using cloud APIs (Google Gemini) resulted in `HTTP 429 Too Many Requests` errors. The free tiers of these services impose strict rate limits that were quickly exceeded.

- **Solution:** The embedding process was moved from the cloud to the local machine. This "Hybrid" approach uses a local model for the "Memory" (embeddings) and a cloud LLM for the "Intelligence" (generation).
- **Implementation:** The `sentence-transformers` library was used with the `BAAI/bge-small-en-v1.5` model, which is an industry standard for local RAG due to its high accuracy on retrieval benchmarks (MTEB). This eliminated network timeouts and API costs for the embedding step.

Performance and Stability Enhancements

- **CPU vs. GPU:** Generating embeddings for thousands of documents on a CPU was time-consuming (~20 minutes). Switching to a GPU runtime (e.g., Google Colab's T4 GPU) reduced this process to under 60 seconds. The `HuggingFaceEmbedding` library automatically detects and utilizes available GPUs.
- **Index Persistence:** To avoid the lengthy embedding process on every run, a save/load mechanism was implemented. The generated vector index is saved to a `storage` directory on the first run and loaded instantly on subsequent runs.
- **Robust API Calls:** To handle intermittent network issues and API failures during the generation step, the code was fortified with `try...except` blocks, automatic retries with exponential backoff, and reduced batch sizes.
- **Dependency Management:** The project ran into numerous version conflicts, particularly between `pandas`, `google-colab`, and various `llama-index` packages.

(`AttributeError: from_tools, AttributeError: 'ReActAgent' object has no attribute 'chat'`). The solution involved creating a stable environment by force-uninstalling conflicting libraries and installing specific, compatible versions.

4. Transition to Agentic Architecture

While the Hybrid RAG system was effective for direct queries, it failed at analytical or multi-step tasks like "List all trains from Ranchi," because the retrieval was limited to the top few results. This led to the adoption of a more advanced "Agentic" architecture. An agent is an LLM that can reason about a problem and decide which "tool" from a provided toolbox is best suited to solve it.

The Two-Tool System: Analyst and Librarian

The agent was given two specialized tools to handle different types of queries:

1. **The Analyst (Pandas Query Engine):** The entire train schedule file was parsed into a structured Pandas DataFrame. This tool allows the agent to perform database-like operations: filtering, counting, and exact lookups. It is used for logical queries, such as finding all trains between two stations or listing all trains at a specific station.
2. **The Librarian (Vector Search Engine):** This tool operates on the vector index of the text data. It is used for descriptive or fuzzy queries, such as "Tell me about the Durgam Cheruvu Express" or "Which train goes via the Konkan route?"

From LlamaIndex to Native Function Calling

The implementation of this agentic system evolved:

- **Initial Attempt (LlamaIndex):** The framework was used to create the tools (`QueryEngineTool`, `PandasQueryEngine`) and the agent (`ReActAgent`). However, this approach was plagued by versioning and dependency issues, making it unstable.
- **Final Solution (Native Gemini):** To eliminate library conflicts, the system was rebuilt using Google's native `google-generativeai` SDK. The tools were defined as standard Python functions, and the model was configured with `enable_automatic_function_calling=True`. This allowed the Gemini model to directly and reliably execute the appropriate Python function based on the user's query.

5. The Final Architecture: Real-Time, Database-Backed Agentic AI

The final phase of the discussion concluded that for a truly robust and relevant system, reliance on a static data file is insufficient. The industry-standard solution is a hybrid system that intelligently sources data.

The Hybrid Data Layer

This architecture separates data based on its volatility:

- **Static Data (Local Database):** Information that rarely changes, such as train numbers, names, routes, and station codes, should be stored in a local database (e.g., SQLite or from a Kaggle CSV). This data is accessed instantly and for free by the "Analyst" tool.
- **Dynamic Data (Real-Time API):** Information that changes constantly, such as live train status, delays, PNR status, and seat availability, must be fetched from an external API (e.g., IRCTC via RapidAPI). A dedicated tool (e.g., `check_live_status`) is created for this purpose.

The Agentic Workflow

The Gemini agent acts as a smart "Router." It parses the user's intent and directs the query to the correct data source:

- A query for "Schedule for the Rajdhani Express" is routed to the local database.
- A query for "Is the Rajdhani Express running late today?" is routed to the live API tool.

This architecture ensures the chatbot is fast and cost-effective for static queries while remaining truthful and up-to-date for dynamic ones. Tools like **LangGraph** were identified as the next step for managing more complex, multi-step conversations and state, solidifying this model as a production-ready, enterprise-grade solution.

4. The Agentic Leap: From Search to Reasoning

The project's scope now evolved beyond a search-and-retrieval system into a true **Agentic** system. An agent, in this context, is a system that uses a language model for reasoning to select the right tool for a given task, rather than being hard-coded to a single retrieval method.

The previous hybrid model, while accurate, could not answer analytical questions like, "List all trains from Ranchi." Vector search is a nearest-neighbor algorithm designed to return a small, fixed number of results (`top_k`); it is mathematically incapable of performing the `SELECT * WHERE . . .` or `GROUP BY` operations of a true database query. To overcome this, we designed a new **two-tool architecture**:

- **The Analyst Tool:** This tool is a Python function that operates on an in-memory **Pandas DataFrame** loaded from our source file. It treats the train data as a structured database and is responsible for handling queries that require exact lookups, counting, filtering, or aggregation across the entire dataset. It provides the precision of a database query engine.
- **The Librarian Tool:** This tool is the evolution of our refined vector search engine. It handles vague, descriptive, or semantic questions where the Analyst tool's strict logic would fail, providing the conceptual understanding needed to interpret ambiguous user intent.

This new paradigm marked a critical leap forward. The system could now intelligently route user queries to the appropriate tool, combining the structured precision of a database with the conceptual understanding of an AI. However, our attempt to implement this within a high-level framework introduced the final—and most frustrating—set of challenges.

5. The Final Architecture: Native Gemini Agent

Our attempt to build the agent within the LlamaIndex framework devolved into an "Implementation Crisis"—a quagmire of opaque `AttributeError` tracebacks, phantom `ModelNotFound` exceptions, and intractable `pandas` version conflicts within the Google Colab environment that brought development to a standstill.

This crisis prompted the final, successful pivot: we abandoned the heavy framework in favor of a **Native Agent built directly on the Google Gemini SDK**. This approach proved to be immediately more stable, transparent, and robust. The final, production-ready system was composed of the following:

1. **Data Ingestion:** The single text file was parsed into two distinct, optimized data structures: a **Pandas DataFrame** to serve as the structured database for the Analyst tool and a **Python Dictionary** mapping train numbers to their raw text schedules for the Librarian tool.
2. **Tool Definition:** Simple, self-contained Python functions were created to act as the agent's tools, such as `find_route`, `get_schedule`, and `search_train_by_name`.
3. **The Agentic Core:** The Gemini model's native function-calling, enabled with `enable_automatic_function_calling=True`, became the heart of the agent. This feature acts as a reasoning loop: it first parses the user's query, **selects** the appropriate Python function from its toolbelt, **extracts** the required arguments (e.g., `source="Thane"`), triggers the function's **execution**, receives the structured data return (e.g., a Pandas Series), and finally **synthesizes** that raw data into a coherent, natural-language response.

This final architecture represents a powerful synthesis: it leverages "old-school" precise coding (Pandas, Dictionaries) for facts and "new-school" AI (Gemini) for natural language understanding and reasoning. This reinforced a critical systems design principle: when core

functionality is at stake, the stability and transparency of a native SDK will always outweigh the abstractions of a high-level framework, especially in a rapidly evolving ecosystem.